UFUG2106 Final Project - Stage 1 Report

1. Objective

The objective is to construct a combinational circuit that implements a given multi-output truth table. The circuit may use only two-input AND gates, two-input OR gates, and one-input NOT gates. Each test case specifies an area and delay for each gate type and a target delay. Our Stage 1 method first constructs a correct circuit, then tries equivalent implementations and selects a candidate using delay and area.

2. Method

2.1 Initial circuit construction

The program processes each output column of the truth table and constructs its function bottom-up. It uses Shannon expansion:

f = (NOT x AND f0) OR (x AND f1),

where f0 and f1 are the cofactors obtained by setting input x to 0 and 1. The truth-table groups are ordered with x0 as the most significant input. The implementation first combines adjacent entries using the least significant input, then continues toward x0 until one signal remains for the output. Constant 0 is signal 0; constant 1 is generated by a NOT gate on signal 0 when needed.

The implementation simplifies branches during construction. For example, if f0 = f1 it returns the common signal, and if (f0, f1) is (0, 1) or (1, 0), it returns x or NOT x respectively. It also simplifies identical gate inputs, operations with constant 0, and double negation.

2.2 Shared circuit representation

The circuit is represented as a directed acyclic graph (DAG). A table keyed by gate type and input signal IDs reuses structurally identical gates, including across different output functions. AND and OR inputs are put in a canonical order to identify equivalent gates with swapped inputs. This is structural sharing; it does not guarantee that every logically equivalent expression is recognized. Gate outputs are created after their inputs, so their IDs respect a topological order.

For each gate, the program computes its arrival time as the maximum arrival time of its inputs plus the gate delay. Inputs and constant 0 have arrival time zero.

2.3 Post-synthesis optimization

The initial circuit is retained as a fallback. The program constructs additional equivalent candidates:

1. It rebuilds the reachable circuit by applying De Morgan's law uniformly to AND gates; another candidate applies it uniformly to OR gates. For example, `a OR b = NOT (NOT a AND NOT b)`.

2. It also constructs a delay-oriented mapping that considers positive and negative forms of intermediate signals. At each gate it compares direct and De Morgan implementations by arrival time. Different gates may therefore use different forms.

The program measures each complete candidate after removing gates not reachable from an output. Its area is the sum of the specified costs of reachable gates, counting each shared gate once. Its delay is the maximum arrival time among the outputs. Candidate selection follows this priority:

- A circuit meeting the target delay is preferred to one exceeding it.

- Among candidates meeting the target, smaller area is preferred, with delay breaking ties.

- If none meets the target, smaller delay is preferred, with area breaking ties.

This is a heuristic search over a limited set of candidates, not a guarantee of globally minimum area or delay. A time-based cutoff can skip or stop optimization; it is not a hard guarantee that the complete program will remain within a case's runtime limit. The program does not explicitly enforce RAM limits.

2.4 Output

After selecting a candidate, the program traverses backward from the output signals to find the gates actually used. It writes only these gates in topological order, with consecutive IDs, followed by the output signal IDs. Unused intermediate gates therefore do not contribute to reported area.

3. Experimental setup and interpretation

The table below reports the 25 supplied Stage 1 cases using `stage1.py` and the independent `verify_python.py` script. The measurements shown are from a local Windows run with Python 3.13.9; elapsed time includes starting the program and excludes the verifier's subsequent correctness checks. Times are rounded to the nearest millisecond and may vary between runs and machines.

The verifier checks the generated gate structure, computes area and delay, and simulates all truth-table assignments in blocks using bit vectors. **Local pass** in this table means that the script reported `correct`: the exhaustive functional check passed and the measured runtime did not exceed the case's specified limit. The local verifier does not measure RAM. Consequently, these results are not official evaluator pass/fail results and do not establish compliance with RAM limits. The Linux evaluator and final Ubuntu server may produce different runtime or resource results.

The target is a delay scoring objective, not a correctness threshold: a locally correct circuit may exceed it. A score is not reported because the scoring formula and official evaluator results are not available here.

4. Results

Case	Local result	Area	Delay	Target delay	Runtime (ms)	Runtime limit (ms)
case_001.txt	Pass	135885	71	77	528	10000
case_002.txt	Pass	40442	53	61	262	10000
case_003.txt	Pass	28206	66	64	300	10000
case_004.txt	Pass	7713	57	57	78	1000
case_005.txt	Pass	1557	36	36	47	100
case_006.txt	Pass	12237	56	60	102	1000
case_007.txt	Pass	207436	87	89	4102	100000
case_008.txt	Pass	15924	56	62	108	1000
case_009.txt	Pass	29267	56	59	162	1000
case_010.txt	Pass	17804	59	61	1142	10000
case_011.txt	Pass	620	47	40	53	100
case_012.txt	Pass	7823	37	40	71	1000
case_013.txt	Pass	11826	67	70	231	10000
case_014.txt	Pass	21600	78	72	2301	10000
case_015.txt	Pass	38556	76	81	711	10000
case_016.txt	Pass	5008	46	47	55	1000
case_017.txt	Pass	573	26	25	70	100
case_018.txt	Pass	5380	33	32	68	100
case_019.txt	Pass	21101	76	73	599	10000
case_020.txt	Pass	824	36	35	54	100
case_021.txt	Pass	69666	72	72	611	10000
case_022.txt	Pass	1286	37	34	58	100
case_023.txt	Pass	454	27	24	47	100
case_024.txt	Pass	192805	83	83	1852	100000
case_025.txt	Pass	217395	71	76	765	10000

All 25 cases passed the local functional and runtime checks. **9 cases exceeded their target delay**: case_003.txt, case_011.txt, case_014.txt, case_017.txt, case_018.txt, case_019.txt, case_020.txt, case_022.txt, case_023.txt. These delay overruns can incur a score penalty but do not by themselves indicate a functional failure.

5. Limitations and next steps

The fixed variable ordering and the small set of candidate rewrites can leave substantial room for improvement. In particular, a locally fast polarity choice does not necessarily minimize the area of the whole shared circuit. The implementation also stores the truth table and may use significant time or memory on large inputs.

Before final submission, run the provided evaluator on Ubuntu for every case, check the reported resource limits, and replace or supplement the local table with the official results. Further work can investigate alternative variable ordering, circuit balancing, and more selective area-aware rewrites while preserving correctness.